Smart Workspace Manager

Complete Python Learning Project - Revised Two-Phase Plan

Version	2.0
Date	18 June 2026
Environment	Python 3.12, venv, pip
Cloud strategy	Azure free-first for personal use and testing

Phase 1: 14-day Streamlit learning product.

Phase 2: FastAPI + React upgrade in up to 7 days.

Free-first Azure deployment for personal use and testing.

Prepared as an independent Python learning project.

Contents

1. Executive Summary

Smart Workspace Manager is one large, independent Python learning project that grows from a Python-first web application into a modern full-stack system. It is designed for a learner who already understands programming concepts through Java and therefore needs practical exposure to Python syntax, Pythonic design, automation, data analysis, machine learning basics, databases, APIs, testing, frontend-backend integration, and cloud deployment.

The project is implemented through one stable repository containing one backend/ folder and one frontend/ folder. Phase 1 uses Streamlit as the active frontend while the reusable Python business logic remains in backend/services/. Phase 2 adds a FastAPI API layer and a React frontend without rewriting the service layer. The Phase 1 Streamlit UI is preserved as a legacy reference rather than permanently deleted.

Final direction: Use Python 3.12 with a normal venv and pip. Do not use Conda. Docker is not required for either phase and is kept as an optional future production exercise. The cloud target is Azure using free plans while the system is only for personal use and testing.

2. Final Decisions and Revision Summary

Decision area	Selected decision	Reason
Project structure	One backend/ and one frontend/ from the beginning	Prevents phase-based duplication and supports direct reuse.
Phase 1 UI	Streamlit	Fast Python-first UI for learning and completing an MVP in 14 days.
Phase 2 UI	React with Vite	Modern frontend that calls FastAPI through HTTP/JSON.
Phase 2 backend	FastAPI	Reuses Phase 1 services and introduces APIs, validation, async concepts, and authentication.
Database	SQLite locally first; Azure SQL Database for deployed Phase 2	Keeps local setup simple while giving a managed cloud database under a free monthly allowance.
Environment	Python 3.12 + venv + pip	Simple, standard, and sufficient for this project.
Conda	Not used	Adds no necessary benefit for the selected libraries and learning goals.
Docker	Optional later	Avoids deployment complexity during the 21-day learning plan.
Cloud	Azure free-first	Appropriate for one-person testing; paid services are introduced only before publication.
Phase 2 alternative	Django removed from the active plan	The project follows one clear path: FastAPI + React.

3. Project Vision, Objectives, and Scope

3.1 Vision

Build one complete web-based workspace application that teaches Python progressively while producing a useful system for uploading, organizing, analyzing, and reporting on files and datasets.

3.2 Objectives

- Learn Python syntax through real application code rather than isolated exercises.
- Practice modular programming, object-oriented design, exceptions, type hints, files, and configuration.

- Build automation for file validation, categorization, organization, cleanup, and logging.
- Use pandas, NumPy, Plotly or matplotlib, and scikit-learn in a controlled data analysis workflow.
- Use SQLite locally, then migrate the deployed system to Azure SQL Database.
- Expose reusable Python services through FastAPI endpoints.
- Build a React frontend that consumes the API.
- Test service, database, and API behavior with pytest.
- Deploy both phases using free Azure services for personal use and testing.

3.3 In Scope

- Dashboard, file upload, file library, automatic file organization, CSV analysis, visualizations, reports, settings, logs, and a basic ML lab.
- Streamlit UI in Phase 1 and React UI in Phase 2.
- FastAPI REST endpoints, Pydantic validation, SQLAlchemy, Alembic, authentication basics, and cloud deployment.
- Small personal test datasets and lightweight model training.

3.4 Out of Scope

- A production multi-tenant SaaS platform during the learning timeline.
- Large-scale data processing, deep learning, GPU workloads, or distributed computing.
- Mobile or desktop native applications.
- Payment processing, enterprise role hierarchies, Kubernetes, or complex microservices.
- Connections to any previous academic, research, organizational, or personal project.

4. Two-Phase Development Strategy

Phase	Duration	Active UI	Backend style	Database	Primary result
Phase 1	14 days	Streamlit	Modular Python services	SQLite	Working learning MVP deployed to Azure App Service F1.
Phase 2	6 build days + 1 buffer day	React/Vite	FastAPI REST API	Azure SQL Database	Professional separated frontend/backend system on free Azure services.

Phase 1 must be complete enough to demonstrate independently. Phase 2 is an upgrade, not a restart. The service layer, utility functions, database models, analysis logic, report logic, and tests remain reusable.

5. System Features and End-to-End Flow

5.1 Major Feature Modules

Module	Purpose
Dashboard	Summary cards, recent uploads, analysis counts, report counts, storage use, and quick actions.
File Upload	Validate file extension and size, save safely, calculate metadata, and create a database record.
File Organizer	Classify files by type or date, rename safely, move files, and create automation logs.
File Library	Search, filter, inspect, download, rename, and delete records and files.
CSV Analyzer	Preview data, inspect data types, detect missing values and duplicates, calculate statistics, and select columns.
Data Cleaning	Apply simple cleaning actions and export a cleaned dataset without overwriting the original.

Module	Purpose
Visualization	Create bar, line, histogram, scatter, and missing-value charts where appropriate.
Report Generator	Create CSV, Excel, and text/HTML summaries; PDF remains an optional extension.
ML Lab	Train one small classification or regression model, display metrics, save the model, and run sample predictions.
Settings and Logs	Manage paths and safe limits; display processing and error logs.
Authentication	Added in Phase 2 for a personal user account and protected API routes.

5.2 End-to-End User Flow

1. The user opens the web application.
2. The dashboard loads system summaries from the database.
3. The user uploads a file.
4. The file is validated, stored safely, and recorded in the database.
5. The organizer classifies the file and places it in the correct managed location.
6. For a CSV file, the user opens the analyzer and requests a preview and summary.
7. The analyzer returns schema information, missing values, duplicates, statistics, and chart-ready results.
8. The user applies optional cleaning actions and exports a cleaned copy.
9. The report service creates a downloadable summary.
10. The ML lab optionally trains a small model on an appropriate dataset.
11. In Phase 2, the same operations are requested by React through FastAPI endpoints.

6. Architecture and Design Principles

6.1 Phase 1 Runtime Flow

Streamlit frontend
frontend/streamlit_app.py and frontend/pages/ collect user input and display results.

|

Service layer
backend/services/ contains file, analysis, report, automation, storage, and ML logic.

|

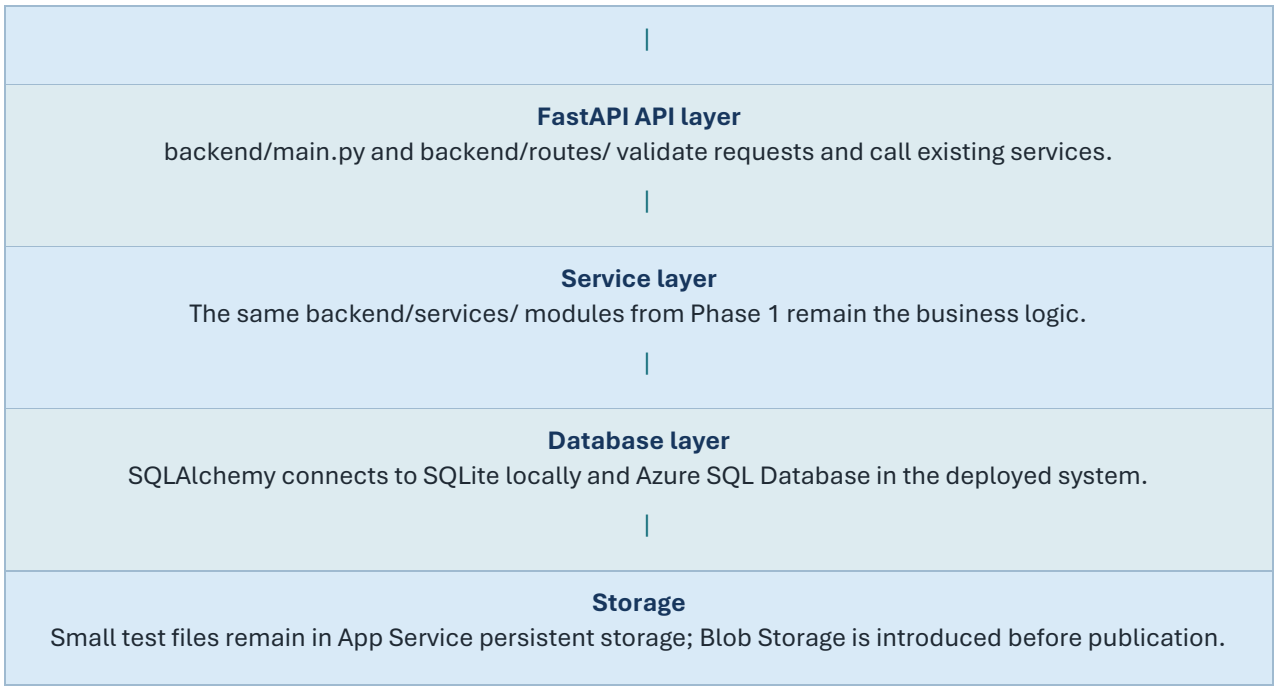
Data access layer
backend/database/ performs SQLite CRUD and returns Python objects.

|

Storage
Local data/ folders during development; /home/data in Azure App Service for small personal test files.

6.2 Phase 2 Runtime Flow

React frontend
Hosted on Azure Static Web Apps Free and sends HTTPS/JSON requests.



6.3 Mandatory Design Rules

- **Thin UI:** Streamlit and React files must not contain core file, analysis, report, or database logic.
- **Reusable services:** Each service accepts normal Python values or file paths and returns structured results.
- **Configuration-driven:** Paths, file limits, database URLs, secrets, and cloud settings come from environment variables or settings.
- **Preserve originals:** Cleaning and organization operations must not silently overwrite original uploads.
- **Small functions:** Prefer testable functions with one clear responsibility.
- **Safe errors:** Log technical details but display understandable messages to the user.
- **One active cloud backend:** The Phase 1 App Service is repurposed for FastAPI in Phase 2 rather than keeping two F1 backend apps running.

7. Phase 1 Specification

Phase 1 is a complete 14-day MVP built primarily in Python. Streamlit acts only as the presentation layer. The service and database code is written with Phase 2 reuse in mind.

7.1 Phase 1 Technology Stack

Area	Selection
Language	Python 3.12
Environment	venv + pip
UI	Streamlit
Database	SQLite; SQLAlchemy recommended from the beginning
Data analysis	pandas and NumPy
Charts	Plotly; matplotlib optional
ML	scikit-learn and joblib
Excel	openpyxl

Area	Selection
Configuration	python-dotenv or pydantic-settings later
Testing	pytest and pytest-cov
Cloud	Azure App Service F1
Cloud file location	Persistent /home/data folders for small test files

7.2 Phase 1 Functional Scope

Feature	Phase 1 requirement
Dashboard	Counts, recent records, quick links, and simple status cards.
Upload	CSV, JSON, TXT, PDF, images, and common documents with size/type validation.
Organizer	Classification, safe names, category/date folders, logs, and duplicate-name handling.
Metadata	File name, stored name, extension, size, path, status, and timestamps.
Analyzer	Preview, data types, missing values, duplicates, summary statistics, and column selection.
Cleaning	Drop duplicates, selected missing-value handling, basic type conversion, and export.
Charts	Basic interactive charts chosen according to column types.
Reports	CSV/Excel/text or HTML summary export.
ML	One small supervised learning workflow on an appropriate sample dataset.
Testing	Unit tests for services plus database integration tests.
Deployment	Streamlit on Azure App Service F1 with a custom startup command.

7.3 Phase 1 Completion Criteria

- The application starts locally through Streamlit using the project venv.
- File upload, metadata storage, organization, listing, download, and delete workflows function.
- CSV analysis and at least three chart types work on small test datasets.
- Cleaned data and an analysis report can be downloaded.
- A lightweight ML demonstration can train, evaluate, save, and predict.
- Tests pass for the main service functions.
- Core logic is not embedded inside Streamlit button blocks.
- The app is deployed to Azure App Service F1 and works within free-tier limits.

8. Phase 2 Specification

Phase 2 converts the system into a modern separated web application. The selected path is FastAPI + React; Django is intentionally excluded from this project plan to keep the seven-day upgrade achievable.

8.1 Phase 2 Additions

Addition	Purpose
FastAPI application	Application factory or main entry point, route registration, CORS, error handlers, and health endpoint.
API routes	Files, analyses, reports, ML, authentication, and settings endpoints.
Pydantic schemas	Request and response models with type validation.

Addition	Purpose
React/Vite frontend	Pages, components, routing, forms, API client, chart rendering, and error states.
Azure SQL Database	Managed relational database for the deployed system under the free monthly offer.
Alembic	Database schema migrations.
Authentication	Personal account login, password hashing, JWT or secure session approach, and protected routes.
API tests	FastAPI TestClient/httpx tests for success, validation, authorization, and errors.
Cloud split	React on Static Web Apps Free; FastAPI on the repurposed App Service F1.

8.2 Phase 2 API Outline

Method	Endpoint	Purpose
GET	/health	Backend status check.
POST	/api/files	Upload a file and create metadata.
GET	/api/files	List and filter files.
GET	/api/files/{id}	Read one file record.
DELETE	/api/files/{id}	Delete the file and metadata safely.
POST	/api/analyses	Analyze a selected CSV.
POST	/api/analyses/{id}/clean	Create a cleaned copy.
POST	/api/reports	Generate a report.
GET	/api/reports	List generated reports.
POST	/api/ml/train	Run the small ML workflow.
POST	/api/auth/login	Authenticate the personal user.
GET	/api/dashboard	Return dashboard summary data.

8.3 Phase 2 Completion Criteria

- React is the active UI and all major functions are requested through FastAPI.
- FastAPI reuses Phase 1 services instead of duplicating business logic.
- The deployed backend uses Azure SQL Database.
- The React frontend is hosted on Azure Static Web Apps Free.
- The FastAPI backend is hosted on Azure App Service F1.
- Login and protected routes work for the personal testing account.
- API tests and service tests pass.
- The legacy Streamlit app remains available in the repository for reference.

9. Azure Free-First Deployment Strategy

Purpose of the free architecture: The system is for one-person learning and testing. The free design is not presented as a production architecture. It intentionally accepts sleep, quota, storage, performance, and no-SLA limitations.

9.1 Phase 1 Cloud Architecture

Browser
The user accesses the Streamlit URL.

|

Azure App Service F1
Runs the Streamlit application and installs Python packages from requirements.txt.

|

SQLite + /home/data
Stores a small local database and small uploaded/processed test files in persistent App Service storage.

|

GitHub
Stores source code and supports deployment.

9.2 Phase 2 Cloud Architecture

Azure Static Web Apps Free
Hosts the React/Vite build for a personal project.

|

Azure App Service F1
Runs FastAPI. The existing Phase 1 App Service is repurposed for the API.

|

Azure SQL Database free offer
Stores users, file metadata, analyses, reports, and logs.

|

App Service /home/data
Stores only small personal test files until the project is prepared for publication.

9.3 Verified Free-Tier Constraints

Service	Relevant limitation or allowance
App Service F1	Shared compute, 60 CPU minutes per day, 1 GB RAM, 1 GB storage, no production SLA.
Azure Static Web Apps Free	Intended for personal projects and supports static hosting and managed SSL.
Azure SQL Database free offer	100,000 vCore seconds and up to 32 GB of data per database each month under the documented free offer.
Operational implication	Keep datasets, ML models, reports, and concurrent work small; delete unnecessary files and monitor quotas.

9.4 Cost-Control Rules

- Create all Azure resources inside one resource group such as rg-smart-workspace-manager.
- Select F1 explicitly for App Service and Free explicitly for Static Web Apps.
- Select the Azure SQL free-offer configuration and set the behavior at the free limit to stop or pause rather than continue into billable use.
- Create budget alerts even when the intended cost is zero.
- Do not create Azure Container Registry, Kubernetes, premium databases, or paid monitoring during the learning plan.
- Delete unused resources and review Cost Analysis after each deployment exercise.
- Treat all free limits as subscription- and region-dependent until confirmed in the Azure portal.

10. Database and Storage Roadmap

10.1 Database Progression

Environment	Database	Reason
Local Phase 1	SQLite	Fast setup and simple debugging.
Azure Phase 1	SQLite file under /home/data	Acceptable only for one-person MVP testing.
Local Phase 2	SQLite through SQLAlchemy	Keeps local setup easy while the ORM matches the cloud architecture.
Azure Phase 2	Azure SQL Database	Managed cloud database for users, metadata, analyses, reports, and logs.
Future publication	Azure SQL paid usage or another selected managed tier	Higher capacity, reliability, and operational controls.

10.2 Core Entities

Entity	Purpose
users	Personal login in Phase 2.
files	Original/stored names, type, size, path or storage key, status, timestamps, and owner.
analysis_jobs	File reference, status, requested options, summary, timestamps, and errors.
reports	Analysis/file reference, report type, path or key, status, and creation time.
automation_logs	Action, target, status, message, and timestamp.
settings	Safe configurable values.
ml_models	Model name, task type, feature/target description, metrics, path, and timestamp.

10.3 Storage Abstraction

All file access should pass through backend/services/storage_service.py. This service hides whether files are stored in the local project, App Service /home/data, or Azure Blob Storage. The initial implementation uses local paths. A future AzureBlobStorageProvider can be added without rewriting upload, analysis, and report logic.

```
Local development:
STORAGE_PROVIDER=local
DATA_ROOT=./data
```

```
Azure free testing:
STORAGE_PROVIDER=local
DATA_ROOT=/home/data

Future publication:
STORAGE_PROVIDER=azure
AZURE_STORAGE_CONNECTION_STRING=...
```

11. Technology Stack and Packages

11.1 Phase 1 Python Packages

Package	Purpose
streamlit	Phase 1 web interface.
pandas	Tabular data loading, cleaning, transformation, and analysis.
numpy	Numerical operations and array support.
plotly	Interactive visualizations.
matplotlib	Optional basic plotting and report images.
scikit-learn	Basic classification/regression workflow.
joblib	Saving and loading small ML models.
SQLAlchemy	ORM and database abstraction.
openpyxl	Excel import/export.
python-dotenv	Local .env configuration.
pytest	Testing.
pytest-cov	Coverage reporting.

11.2 Phase 2 Python Additions

Package	Purpose
fastapi	REST API framework.
uvicorn[standard]	Local ASGI server.
gunicorn	Linux process manager for Azure App Service.
pydantic-settings	Typed configuration.
python-multipart	File upload handling.
alembic	Schema migrations.
pyodbc	Azure SQL connectivity through SQLAlchemy.
PyJWT	JWT token creation and validation.
pwdlib[argon2]	Password hashing.
httpx	API client and API tests.
pytest-asyncio	Async test support.

11.3 React Packages

Package	Purpose
react and react-dom	Frontend runtime.
vite	Development server and production build.
react-router-dom	Client-side routing.
axios	API calls.
react-plotly.js or another chart wrapper	Frontend charts.
vitest and React Testing Library	Optional frontend tests.

11.4 Environment Decision

Use venv: Create the environment with py -3.12 -m venv smart-workspace. Conda is unnecessary. Docker is deliberately excluded from the required work plan and may be added after the non-containerized Azure deployment works.

12. Security, Testing, and Quality Plan

12.1 Security Basics

- Validate extensions, MIME information where possible, file sizes, and generated file names.
- Never trust a user-provided path; create server-controlled storage paths.
- Keep .env, database credentials, JWT secrets, and connection strings out of Git.
- Hash passwords; never store plain-text passwords.
- Restrict CORS to the deployed frontend URL in Phase 2.
- Use parameterized ORM queries and schema validation.
- Limit uploaded file size and reject unsupported formats.
- Do not expose internal filesystem paths in API responses.

12.2 Test Levels

Test level	Scope
Unit tests	Validators, safe naming, category detection, cleaning functions, summary calculations, and report helpers.
Database integration	CRUD, relationships, transaction behavior, and migration checks.
Service integration	Upload + metadata + organization; analysis + report generation.
API tests	Success, validation failure, not found, unauthorized, and server error behavior.
Manual UI tests	Navigation, forms, upload states, chart rendering, and downloads.
Cloud smoke tests	Health endpoint, one upload, one analysis, one report, and one login after deployment.

12.3 Quality Rules

- Use clear names, type hints, docstrings for public functions, and small focused modules.
- Use logging instead of scattered print statements.

- Commit after meaningful milestones and tag Phase 1 and Phase 2 releases.
- Keep requirements explicit and update README run/deploy commands.
- Preserve sample data separately from user-uploaded test data.
- Perform a final clean setup test from a fresh venv.

13. Python Concepts and Technologies Covered

Area	Coverage
Core syntax	Variables, types, strings, numbers, booleans, None, conditions, loops, functions, imports, and modules.
Collections	Lists, dictionaries, tuples, sets, comprehensions, sorting, filtering, and iteration.
Functions	Parameters, defaults, keyword arguments, return values, scope, lambdas where appropriate, and callbacks.
OOP	Classes, objects, constructors, instance methods, composition, inheritance only where justified, and dataclasses.
Errors	Built-in exceptions, custom exceptions, try/except/finally, validation, and error propagation.
Files	pathlib, reading/writing, binary files, CSV, JSON, Excel, safe renaming, moving, and deletion.
Advanced Python	Type hints, decorators through routes, context managers, generators/streaming where useful, and async/await in FastAPI.
Automation	Batch organization, logs, cleanup rules, scripts, and scheduled concepts.
Data science	DataFrames, missing values, duplicates, summaries, transformations, charts, and simple preprocessing.
Machine learning	Feature/target selection, train/test split, fitting, prediction, metrics, persistence, and limitations.
Databases	CRUD, models, sessions, relationships, transactions, migrations, and database URLs.
Web development	Streamlit state, REST, HTTP methods, status codes, JSON, validation, CORS, auth, and frontend integration.
Testing	Fixtures, assertions, mocking where needed, API tests, coverage, and regression prevention.
Cloud	Azure deployment, startup commands, environment variables, quotas, managed database, and static hosting.

14. Limitations and Topics Outside Scope

Area	Reason
Desktop GUI	Tkinter, PyQt, and Kivy require a separate project.
Mobile development	Professional mobile development is outside the selected stack.
Game development	Pygame and real-time game loops are not represented.
Deep learning	PyTorch/TensorFlow and large neural models require a separate focused project.
Big data	Spark, Kafka, Airflow, and distributed processing exceed the project scope.
Advanced cloud	Kubernetes, Terraform, service mesh, and multi-region architecture are not needed.
Heavy background processing	Celery/Redis can be introduced later but are not suitable for the free-first timeline.
Enterprise security	The project covers essential app security, not penetration testing or compliance.
Advanced React	React is learned enough to operate this frontend, not as a complete frontend-specialist curriculum.
Python internals	CPython implementation, C extensions, the GIL in depth, and memory internals remain separate topics.

15. Deliverables and Completion Criteria

ID	Deliverable
D1	Git repository with stable combined structure.
D2	Working Streamlit Phase 1 application.
D3	Reusable backend service and database layers.
D4	File management, analysis, reporting, and basic ML features.
D5	pytest suite and test report.
D6	Azure App Service F1 Phase 1 deployment.
D7	FastAPI backend and documented endpoints.
D8	React/Vite frontend.
D9	Azure SQL database and migrations.
D10	Static Web Apps Free frontend and App Service F1 API deployment.
D11	README, project document, folder document, work plan, API notes, screenshots, and known limitations.

16. Risks, Constraints, and Mitigations

Risk	Impact	Mitigation
Scope too large	Features may not finish in 21 days.	Use strict MVP criteria; treat PDF, advanced ML, jobs, and Docker as optional.
Logic mixed into UI	Phase 2 becomes a rewrite.	Require all important operations to live in backend/services/.
Free App Service quota	App stops after quota exhaustion.	Use small datasets, avoid repeated training, and repurpose rather than duplicate F1 apps.
Cloud storage limit	Uploads fill the 1 GB quota.	Set file-size limits, clean test files, and move to Blob Storage only before publication.
Azure SQL configuration	Unexpected billing or connectivity.	Select the free offer, stop at free limits, test the connection early, and monitor cost.
React learning overhead	Phase 2 exceeds seven days.	Use a simple component library/layout and implement only existing Phase 1 features.
Authentication complexity	Delays final integration.	Implement one personal user and basic protected routes; defer roles and password reset.
Package/version conflict	Environment becomes unstable.	Use Python 3.12, venv, pinned dependencies after setup, and a clean install test.

17. Future Publication Upgrade Path

Area	Free testing state	Publication upgrade
Compute	App Service F1	Paid Basic/Standard App Service plan with Always On and higher resources.

Area	Free testing state	Publication upgrade
Frontend	Static Web Apps Free	Keep Free if sufficient or move to Standard for production requirements.
Database	Azure SQL free allowance	Paid capacity, backups, monitoring, and performance tuning.
File storage	App Service /home/data	Azure Blob Storage with private containers and lifecycle rules.
Secrets	App settings	Azure Key Vault and managed identity.
Monitoring	Basic logs	Application Insights, alerts, dashboards, and retention.
Jobs	Synchronous processing	Azure Functions or a worker/queue system for long jobs.
Deployment	Direct/GitHub deployment	Controlled CI/CD, staging, release approvals, and optional Docker.

18. References

Official sources checked for the revised cloud decisions:

1. Azure App Service for Linux pricing and F1 limits: https://azure.microsoft.com/en-us/pricing/details/app-service/linux/

2. Azure SQL Database free offer: https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer?view=azuresql

3. Azure SQL Database free-offer FAQ: https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer-faq?view=azuresql

4. Azure Static Web Apps hosting plans: https://learn.microsoft.com/en-us/azure/static-web-apps/plans

5. Deploy Python web apps to Azure App Service: https://learn.microsoft.com/en-us/azure/app-service/quickstart-python

6. Configure Python apps on Azure App Service: https://learn.microsoft.com/en-us/azure/app-service/configure-language-python

7. Streamlit documentation: https://docs.streamlit.io/

8. FastAPI documentation: https://fastapi.tiangolo.com/

9. SQLAlchemy documentation: https://docs.sqlalchemy.org/

10. pandas documentation: https://pandas.pydata.org/docs/

11. scikit-learn documentation: https://scikit-learn.org/stable/

12. pytest documentation: https://docs.pytest.org/